Assumption 2 simply states that each query is clustered the same number of times with all its' big inner products along the different hashing rounds.

Assumption 3 (Effectiveness of LSH clustering)**.** *There is a small constant H, which denotes the number of hashing rounds, such as:*

$$\forall k \in \mathcal{K}: \; w_q \neq 0 \Rightarrow \exists n: \; 1 \leq n \leq H \; \wedge \; k \in \mathcal{K}_{q_n}.$$

The latter assumption states that we need a small number of hashing rounds H to catch all big inner products of a given query.

We state the following theorem:

Theorem 2. *If Assumptions 1, 2, 3 hold, then our approximation algorithm is exact.*

Proof of Theorem 2. With our merging scheme (Equation 15), the attention output is:

$$o'_q = \sum_{h=1}^{H} \sum_{k \in \mathcal{K}_{h_q}} \left(\frac{\sum_{k' \in \mathcal{K}_{h_q}} e^{q \cdot k'}}{\sum_{n=1}^{H} \sum_{k' \in \mathcal{K}_{n_q}} e^{q \cdot k'}} \cdot \frac{e^{q \cdot k}}{\sum_{k' \in \mathcal{K}_{h_q}} e^{q \cdot k'}} \right) \cdot v_k = \sum_{h=1}^{H} \frac{\sum_{k \in \mathcal{K}_{h_q}} e^{q \cdot k} \cdot v_k}{\sum_{n=1}^{H} \sum_{k' \in \mathcal{K}_{n_q}} e^{q \cdot k'}} \tag{16}$$

Under Assumption 1, the dense attention output for this query is the vector:

$$o_q = \sum_{k \in \mathcal{K}_q} \frac{e^{q \cdot k}}{\sum_{k' \in \mathcal{K}_q} e^{q \cdot k'}} \cdot v_k$$

where K_q is the set of keys k_i for query q for which $w_i \neq 0$.

Under Assumption 3, all keys that have big inner product with a given query q are clustered with that query, at least one time. Also, under Assumption 2, all these keys are clustered the same amount of times with each query. We will denote the amount of a query is clustered with each one of its' big inner products with N_q. It holds that:

$$\sum_{n=1}^{H} \sum_{k' \in \mathcal{K}_{n_q}} e^{q \cdot k'} = N_q \cdot \sum_{k' \in \mathcal{K}_q} e^{q \cdot k'} \tag{17}$$

By substitution in Equation 17, we get:

$$o_q = \frac{\sum_{n=1}^{H} \sum_{k \in \mathcal{K}_{h_q}} e^{q \cdot k} \cdot v_q}{N_q \cdot \sum_{k' \in \mathcal{K}_q} e^{q \cdot k'}} \tag{18}$$

Under Assumptions 1, 2 small inner products get a zero-score and all big inner products are clustered N_q times each. Thus, we can write for the nominator: $\sum_{n=1}^{H} \sum_{k \in \mathcal{K}_{h_q}} e^{q \cdot k} \cdot v_q = N_q \sum_{k \in \mathcal{K}_q} e^{q \cdot k'}$.

Substituting to Equation 18, we get:

$$o'_q = \sum_{k \in K_q} \frac{e^{q \cdot k}}{\sum_{k' \in \mathcal{K}_q} e^{q \cdot k'}} v_k = o_q$$

□

In this section, we explained in detail our merging scheme. We also showed that under certain assumptions on the data, this scheme leads to exact approximations of dense attention output. We fully understand that the assumptions are far too tight to hold in practice and since distortion is introduced. However, as we demonstrated in the Experiments section, the distortion is negligible even for large memory reductions, since SMYRF can perform on par (or even better, e.g. GLUE) with dense attention, especially on downstream Natural Language Processing tasks, using a fraction of the original memory.

14 Complexity analysis and speedups

In the paper, we presented shortly the complexity of our algorithm. In this section, we explain it in more detail and we also include speed plots that demonstrate the effectiveness of SMYRF for long sequences.

14.1 Complexity Analysis

For the complexity analysis, we assume for simplicity that $|\mathcal{Q}| = |\mathcal{K}| = N$, i.e. the number of available queries is equal to the number of available keys.

We run the algorithm H times (i.e. rounds of LSH). Each run has two stages:

- Clustering in L clusters (of equal size). For clustering, we hash all points with LSH which requires complexity $O(N)$ and then we sort points based on their hash, which requires complexity $O(N \cdot \log N)$. Overall, the complexity is $O(N \cdot \log N)$.
- Within clusters attention. Attention within each cluster has quadratic cost with respect to the cluster size. Each cluster has size $\frac{N}{L}$, so the complexity of attention in a single cluster is $O(\frac{N^2}{L^2})$. We have L such clusters, and thus the overall complexity is $O(\frac{N^2}{L})$.

The total complexity is: $O\left(H \cdot N \cdot \log N + H \cdot \frac{N^2}{L}\right)$. We choose $L = O(N)$, i.e. each query attends to a small constant number of keys. We obtain complexity: $O(H \cdot N \cdot \log N)$.

14.2 Speedups

In this subsection, we present two speed plots to demonstrate the speed effectiveness of SMYRF for large sequences. The first plot, Figure 8, shows elapsed time for SMYRF-BERT (base) GPU inference for various batch-sequence length configurations. In all these experiments batch size $\times\ N = 65$K, where N denotes the sequence length. We underline that SMYRF has (almost) constant speed in all these configurations while the speed of dense attention decreases rapidly us the sequence length increases. Notably, SMYRF is already faster than dense attention in sequence length 1024 tokens. The second plot, Figure 9, shows seconds per iteration for SMYRF-BERT (base) GPU inference for various hashes-cluster configurations. In all these experiments, batch size is fixed to 1. As shown, all different configurations significantly outperform (in terms of speed) dense attention as the sequence length increases.

15 Experimental details

15.1 Natural Language Processing experiments

In this section, we provide some details about the experimental settings for the Natural Language Processing experiments.

15.1.1 IMDB

IMDB [52] contains 25,000 train and 25,000 dev labeled movie reviews. The task is to identify if a given movie review is positive or negative. The average sentence length in IMDB is 300 tokens and the 95th percentile of context length is 705 tokens. In our experiments, we truncated/padded all sentences to 512 tokens. For all our experiments, we trained for 3 epochs, with batch size 8. We used Adam [76] as our optimizer with learning rate $3 \cdot 10^{-5}$. The dataset is available publicly in this link: https://ai.stanford.edu/ amaas/data/sentiment/. The experiments on IMDB run on a single GPU provided by Google Colab.

15.1.2 GLUE

GLUE [25] is a standard multitask benchmark for Natural Language Processing. For a full description of tasks, dataset statistics and files, please refer to the official website: https://gluebenchmark.com/. Following previous literature (e.g. [9, 6, 5, 24]), for our GLUE experiments we truncate/pad all

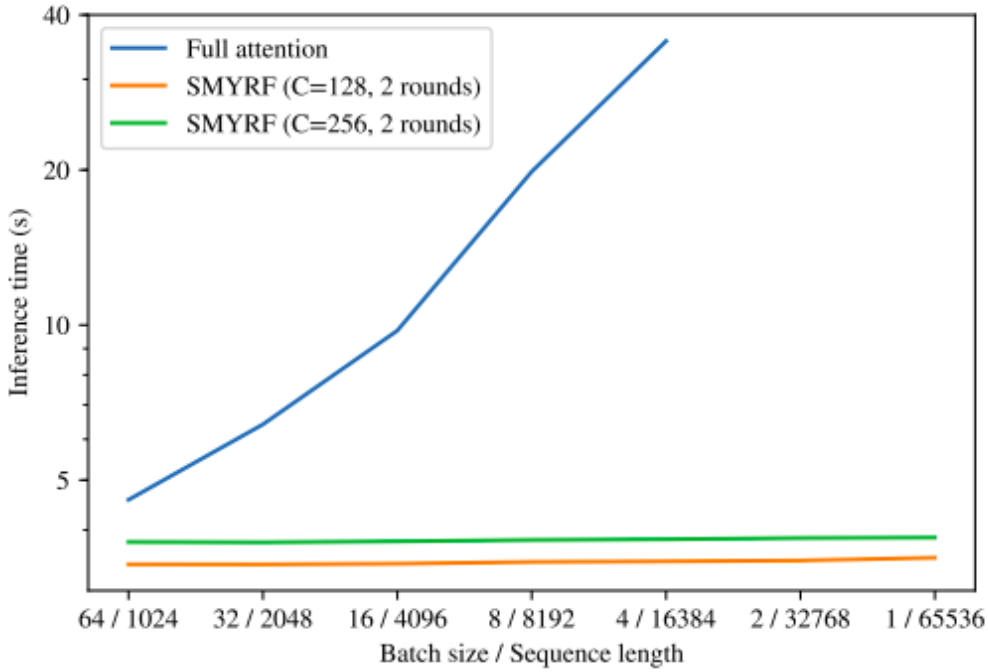

Figure 8: Elapsed time for SMYRF-BERT (base) GPU inference for various batch-sequence length configurations. Elapsed time for SMYRF is almost constant for all configurations. Elapsed time for dense attention worsens a lot as we increase the sequence length.

input sentences to 128 tokens. For GLUE, we trained for 3 epochs at batch size 16, warming up for 10% of the total training time. The learning rate was selected via grid search among the values $\{5 \cdot 10^{-5}, 3 \cdot 10^{-5}, 2 \cdot 10^{-5}\}$. We run the GLUE experiments on TPUs.

15.2 Training SMYRF-BigGAN on Celeba-HQ

In the paper we presented results for training SMYRF-BigGAN from scratch on Celeba-HQ [29]. As explained, we trained on Celeba-HQ (and not ImageNet [37]) in order to save computational resources. In this section, we provide the details for these experiments. First of all, as the name suggests, we used as the underlying model, BigGAN [1]. For our experiments we disabled BigGAN's hierarchical latent codes, shared embeddings and skip-z connections since Celeba-HQ has one single class (humans) and these architectural choices were introduced to model multiple classes (e.g. 1000 classes on ImageNet). We also found that for the single-class Celeba-HQ we didn't have to use very large batch sizes for stable training. For all our experiments, we used batch size 32. Following the BigGAN paper, we used Two Time Scale Update Rule (TTUR) [39] with Adam [76] optimizer, $\text{G}_{\text{lr}} = 2 \cdot 10^{-4}$, $\text{D}_{\text{lr}} = 5 \cdot 10^{-5}$, $\beta_1 = 0$ and $\beta_2 = 0.999$.

BigGAN for resolutions $\{128 \times 128, 256 \times 256\}$, is trained with a single attention layer at resolution 64×64. The authors mention that they stick attention to low resolution to save computational resources. We take advantage of SMYRF's reduced memory requirements to train with attention at resolution 128×128 and 256×256. For both experiments, we remove the dense attention layer and we add a SMYRF attention layer. Since our goal is to demonstrate the ability of SMYRF layers to train successfully from scratch, there is no reason to use higher (image) resolutions than the attention resolution and thus SMYRF is the final layer (before Tanh [77]) in the architecture. In other words, we train on image resolutions $128 \times 128, 256 \times 256$ respectively. Training on resolution 128×128 has the side-benefit that we can compare directly with the original BigGAN model (with dense attention at 64×64). As we demonstrated in the Experiments section of the paper, moving attention

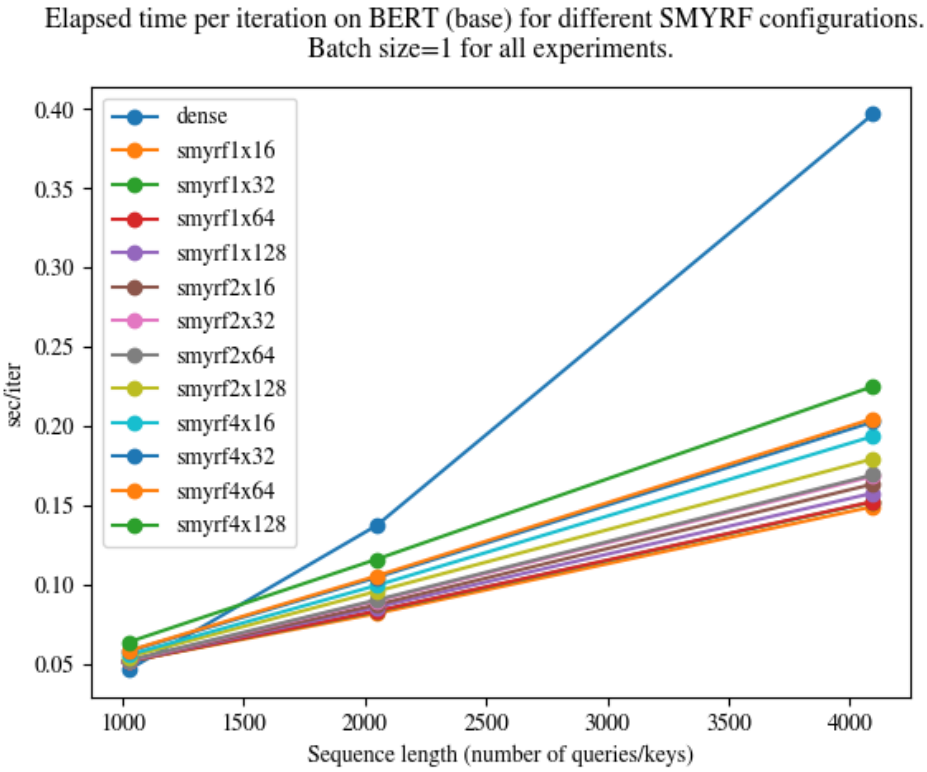

Figure 9: Seconds/iteration for SMYRF-BERT (base) GPU inference for various hashes-cluster configurations. In all experiments batch size is fixed to 1. SMYRF has approximate the same speed with dense attention at 1024 tokens. However, as the number of tokens increases, SMYRF is significantly faster than dense attention.

from 64×64 to 128×128 can lead to $\approx 4\%$ FID [39] improvement after 120K training steps[6]. We present random generated images from SMYRF-BigGAN with attention at resolution 256×256 at Figure 10. As explained in the Things that did not work section, training with SMYRF from scratch is harder as the sequence length increases. The main reason is that during the early stages of training attention maps are not sparse and thus our approximation's algorithm output is not close to the dense attention output. We noticed that the overall performance of SMYRF-BigGAN-256 is lower compared to SMYRF-BigGAN-128 and the generated images seem slightly less realistic. Despite the aforementioned shortcomings, this experiment demonstrated that it is possible to successfully train an attention GAN with attention at 256×256 resolution on a *single* TPUv3-8 device. The training at 128×128 resolution lasts approximately 1.5 day and at 256×256 resolution approximately 2 days.

16 Things that did not work

In this section, we discuss some negative results we encountered in the process of writing this paper. Our goal is to share our experience with the research community about the observed shortcomings of some approaches so that future research can re-formulate them, reject them or even contradict our findings. We also include some suggestions on potential ways to alleviate such problems that we did not have the time to explore in this paper.

[6]We note that in order to save computational resources we stopped training for both models on 120K iterations, before mode-collapse. That means that further training could possibly lead to even better FID scores for both models.